UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 5: Vectors and Collections

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

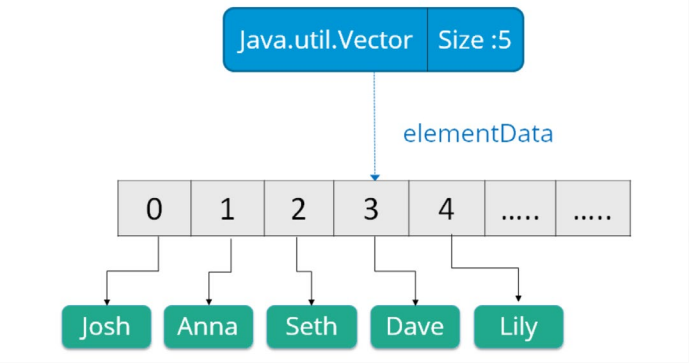
Contents

- **Array List:**
  - Creating and Adding Items
  - Accessing and Removing Items
  - Methods for ArrayList Class
- **Vector**
  - Introduction and Examples
  - Methods for Vector Class

Introduction to `ArrayList` Class and `Vector` Class

INTRODUCTION

- Similar to an array, an `ArrayList` **allows object storage**
- Unlike an array, an `ArrayList` object:
  - **Automatically expands** when a new item is added
  - **Automatically shrinks** when items are removed
- It requires: `import java.util.ArrayList;`
- A vector is basically the **same as an `ArrayList`**, but `Vector` methods are **synchronized** for **thread safety**
- It requires: `import java.util.Vector;`



Source: https://www.edureka.co/blog/java-arraylist/

Creating an `ArrayList` and `add()` Method

ARRAY LIST

- **Create** an `ArrayList` object with no-args constructor:

```
ArrayList townList = new ArrayList();
```

- **Add element** to the `ArrayList`, use the `add()` method:

```
townList.add("Kangar");
townList.add("Alor Setar");
```

- To get the current **size**, call the `size()` method:

```
townList.size();  // returns 2
```

Accessing & Removing Items in an `ArrayList`

- To **access** items in an `ArrayList`, use the `get()` method as follows:

```
townList.get(1); // 1 is the index of the item to get
```

- A **loop** is used in the following statement to access every element in the `ArrayList` named `townList`.

```
for(int i=0; i < townList.size(); i++)
        System.out.print(townList.get(i)+" ");
```

- To **remove** items in an `ArrayList`, use the `remove` method

```
townList.remove(1); //This statement removes the second item
townList.remove("Penang"); // This statement removes the
                           // item with the value "Penang"
```

Adding and Replacing Existing Items Using Two Argument Method

ARRAY LIST

- The `ArrayList` class' `add()` method with **one argument** adds new items to the **end of the ArrayList**
- To insert items at a **location of choice**, use the `add()` method with **two arguments**:

```
// This statement inserts the String "Shah Alam" at index 6
townList.add(6, "Shah Alam");
```

- To **replace** an existing item, use the `set()` method:

```
// This statement replaces the value at index 1 with "Muar"
townList.set(1, "Muar");
```

Using `toString()` method and `ArrayList`

ARRAY LIST

- The `ArrayList` class' **`toString()`** method returns a **string representing all items** in the `ArrayList`

```
System.out.println(townList);
```

- This statement yields : `[Muar, Alor Setar]`
- An `ArrayList` has a capacity, which is the number of items it can hold without increasing its size.
- The **default capacity** of an `ArrayList` is 10 items.
- To **designate a different capacity** that will grow large, use a parameterized constructor [Note: It does not limit the size, but avoid automatic resizing that can improve performance]:

```
ArrayList list = new ArrayList(100);
```

Using a Cast Operator with the `get()` Method

ARRAY LIST

- An **`ArrayList`** object is **not typed if not specified as generics**
- To retrieve items from an **`ArrayList`**, we must **cast the item** to the appropriate type e.g.

```
ArrayList nameList = new ArrayList();
townList.add("Kluang"); // Inserts an item
String str = (String)townList.get(0);
```

- Try **`get()`** without the cast to see the effect.

Using `ArrayList` as Generics Data Type

ARRAY LIST

- Can create a **type-safe `ArrayList`** object by using **generics**, compiler knows the type and helps catch errors early.
- Example an **`ArrayList`** object for String:

```
ArrayList<String> nameList = new ArrayList<String>();
```

- Since Java 7, can use the **diamond operator** <> without stating the type on the right-hand side of a variable declaration e.g.

```
ArrayList<String> nameList = new ArrayList<>();
```

- The e.g. above creates a type-safe `ArrayList` that only accepts String objects.
- The **`get()`** method no longer requires casts to work.

`ArrayList` Methods

ARRAY LIST

- **`add(Object o)`**: add new object into `ArrayList` [Note: Modern usage uses generics: `add(E e)` where E is the element type i.e. not necessarily object]
- **`get(int index)`**: retrieves object reference from `ArrayList` index position
- **`size()`**: returns `ArrayList` size i.e. number of elements in the list
- **`remove(int index)`**: removes the element at the specified position in this list. Shifts any subsequent elements to the left and returns the element that was removed from the list.
- **`indexOf(Object o)`**: finds the index in this list of the first occurrence of the specified element
- **`clear()`**: removes all elements for the list

Example 1

ARRAY LIST

- Create **`ArrayList`** to store `Circle` objects
- Create both classes in the same Java file

```java
import java.util.ArrayList;

class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

Example 1 (cont.)

ARRAY LIST

```java
public class CircleArrayList {
    public static void main(String[] args) {
        // Create an ArrayList to store Circle objects
        ArrayList<Circle> circleList = new ArrayList<>();

        // Add Circle objects to the list
        circleList.add(new Circle(2.0));
        circleList.add(new Circle(3.5));
        circleList.add(new Circle(1.2));

        // Display the radius and area of each circle
        for (int i = 0; i < circleList.size(); i++) {
            Circle c = circleList.get(i);
            System.out.println("Circle " + (i + 1) + ":");
            System.out.println("  Radius = " + c.getRadius());
            System.out.println("  Area = " + c.getArea());
            System.out.println();
        }
    }
}
```

Example 2: Using `ArrayList` methods

ARRAY LIST

```
import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        // Create an ArrayList of Strings
        ArrayList<String> names = new ArrayList<>();

        // 1. add(Object o): Add new elements
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");
        names.add("Diana");

        // 2. get(int index): Get element at index 2
        System.out.println("Element at index 2: " + names.get(2)); // Charlie

        // 3. size(): Get the size of the list
        System.out.println("Size of list: " + names.size()); // 4
```

Example 2 (cont.)

ARRAY LIST

```
        // 4. remove(int index): Remove element at index 1 (Bob)
        String removedName = names.remove(1);
        System.out.println("Removed: " + removedName); // Bob

        // 5. indexOf(Object o): Find index of "Diana"
        int index = names.indexOf("Diana");
        System.out.println("Index of Diana: " + index); // 2

        // 6. clear(): Remove all elements
        names.clear();
        System.out.println("List after clear(): " + names); // []

        // Confirm list is empty
        System.out.println("Is the list empty? " + names.isEmpty()); // true
    }
}
```

Introduction to `Vector` Class

VECTOR

Size of an array is fixed for the duration of the execution program.
Vector is a more flexible data structure which allows its size to be changed.

- A `Vector` class is part of the `java.util` package
- It can **dynamically** shrink or grow as needed
- A **data element** can be **inserted into or removed from** any location of a vector
- The elements of a vector **must be objects** (cannot be primitive types - to store a primitive type, we need to use appropriate wrapper classes).
- A `Vector` class is basically the **same as an `ArrayList`**, but `Vector` methods are **synchronized** for **thread safety**.

Library: `java.util.Vector`

VECTOR

- **Regular arrays** are limited to their initial size. They **cannot grow or shrink** (*static*)
- The **`java.util.Vector`** class implements an array of objects that can grow as needed (*dynamic*)
- The size of a vector does not have to be declared: it **grows and shrinks as needed**
- There are methods to **add, retrieve, insert** in the middle of the vector, or **remove elements**

Vectors: API Documentation (partial listing)

VECTOR

```
public class Vector extends Object implements Cloneable, Serializable
{
  // constructors
   public Vector (int initialCapacity);
   public Vector();
  // Instance methods for different functions
  public final void addElement (Object obj); - insert element at next free location
  public final void insertElement (Object obj, int index); - insert element at certain location
  public final int  capacity();   - return capacity of vector
  public final Object elementAt(int index);- return object stored at the position 'index'
  public final Object firstElement(); - return first element (object at vector position 0)
  public final int  indexOf(Object elem); - return index of the given element in vector
  public final int  lastIndexOf(Object elem); - return last index of the given element in vector
  public final Object lastElement(); - return last element (object at position size-1)
  public final synchronized boolean removeElement(Object obj); - removes specified object
  public final synchronized void removeAllElements(); - removes all objects
  public final synchronized void removeElementAt(); - removes object at certain index
  public final int size(); - return number of objects stored in the vector
  public final void trimToSize(); - reduce capacity of the vector to the number of elements
}
```

Allows specification of initial capacity of vector

VECTOR

`Vector`: Import Statement and Declaration

- Use vector to store a **collection of data** that contains texts e.g. faculty names
- **Import** statement: `import java.util.Vector;`
- **Declare** a `Vector` object of faculty:

```
Vector faculty = new Vector();
//syntactically correct, but not recommended in modern Java
```

- Since Java 7, can use the **diamond operator** <> for **generics** data type, which is **type-safe**, modern best practice :

```
Vector<String> faculty = new Vector<>();
```

- A `Vector` of size 0 is created to store String type.
- **Note:** No casting is needed when retrieving elements for generics data type

Without Generics (Raw Types) vs. With Generics

VECTOR

```java
import java.util.Vector;

public class Main {
    public static void main(String[] args) {
        Vector employees = new Vector(); // raw type
        employees.add("Alice");
        employees.add("Bob");

        String name = (String) employees.get(0); // must cast
        System.out.println(name);
    }
}
```

```java
import java.util.Vector;

public class Main {
    public static void main(String[] args) {
        Vector<String> employees = new Vector<>(); // type-safe
        employees.add("Alice");
        employees.add("Bob");

        String name = employees.get(0); // no cast needed
        System.out.println(name);
    }
}
```

VECTOR

Example 1

- **Add** new element to a **data collection**:

```
faculty.addElement("FC");      //or faculty.add("FC");
faculty.addElement("FS");      //or faculty.add("FS");
faculty.addElement("FKE");     //or faculty.add("FKE");
faculty.addElement("FAB");     //or faculty.add("FAB");
```

- The `Vector` will contain **four elements** in location 0, 1, 2 and 3.
- Display the **content** of vector:

```
System.out.println(faculty);
```

- Display the **size** of the vector:

```
System.out.println(faculty.size());
```

Example 1 (Cont.)

- To **insert** new element at the third element of the vector:

```
faculty.insertElementAt("FPPSM", 2);
//"FPPSM" is stored in between "FS", and "FKE"
```

- Display the **content** of vector:

```
System.out.println(faculty);
```

- Display the **size** of the vector:

```
System.out.println(faculty.size());
```

- To **remove** element with content "FKE" in the vector:

```
faculty.removeElement("FKE");
```

Example 1 (Cont.)

- To **remove** element at the third element of the vector:

```
faculty.removeElementAt (2);
```

  - "**FS**" will be deleted.
- To **display** the **content** of vector at the specified element:

```
System.out.println (faculty.elementAt(1));
```

  - "**FPPSM**" will be displayed.
- To **change** the value of specific element of a vector

```
faculty.setElementAt("FKSG",2);
```

  - The content of element three will be replaced by "**FKSG**".

VECTOR

Vector Methods: `add`, `addAll`, `addElement`

All Methods | Instance Methods | Concrete Methods

Modifier and Type	Method	Description
boolean	`add(E e)`	Appends the specified element to the end of this Vector.
void	`add(int index, E element)`	Inserts the specified element at the specified position in this Vector.
boolean	`addAll(Collection<? extends E> c)`	Appends all of the elements in the specified Collection to the end of this Vector, in the order that they are returned by the specified Collection's Iterator.
boolean	`addAll(int index, Collection<? extends E> c)`	Inserts all of the elements in the specified Collection into this Vector at the specified position.
void	`addElement(E obj)`	Adds the specified component to the end of this vector, increasing its size by one.

https://docs.oracle.com/javase/8/docs/api/java/util/Vector.html

Note:

- If **thread safety** is required e.g. multiple threads access shared resources, then use **`addElement()`**
- If it is **not a concern, `add()` might be slightly more efficient** due to the absence of synchronization overhead.
- In addition, `add()` allows for checking the success of the operation via its boolean return value.

Vector: Example 2

VECTOR

```java
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {

        // Create a Vector of Strings
        Vector<String> fruits = new Vector<>();

        // 1. Add elements to the vector
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");

        // Insert at a specific position (e.g., index 1)
        fruits.add(1, "Mango");  // Now: Apple, Mango, Banana, Cherry

        // 2. Display contents
        System.out.println("Fruits in the vector:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }
```

VECTOR

Example 2 (Cont.)

```
        // 3. Display size of the vector
        System.out.println("Size of the vector: " + fruits.size());  // 4

        // 4. Remove an element by index
        String removed = fruits.remove(2);  // Removes "Banana"
        System.out.println("Removed: " + removed);

        // 5. Modify an element (change "Cherry" to "Peach")
        fruits.set(2, "Peach");  // index 2 was "Cherry", now it's "Peach"

        // 6. Final content display
        System.out.println("Updated fruits in the vector:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}
```

Wrapper Classes for Primitive Types

VECTOR

- Example of use to store in a vector where **primitive types** cannot be stored:

```
double x = 13.5;
Vector numbers  = new Vector();
numbers.addElement(new Double(x));
double y = ((Double) numbers.elementAt(0)).doubleValue();
```

Wrapper Classes for Primitive Types (Cont.)

VECTOR

- Assume vector `v` contains a **collection of objects**, some of which are type Double.
- The statements below, **store** some objects in vector `v`.
- The **loop** (see next slide) adds only the numbers wrapped in the type `Double` objects and displays their sum (50.5).

```
Vector v = new Vector();
OR Vector<Double> v = new Vector<>(); // generics data type
v.addElement("Jamil");  // first element is a string
OR v.add("Jamil"); //generics data type, thread safety is not a concern
v.addElement(new Double(15.0)); // second element is type Double
v.addElement(new Double(35.5)); // third element is type Double
v.addElement(new Integer(100)); // fourth element is type Integer
```

Wrapper Classes for Primitive Types (Cont.)

VECTOR

```
double sum = 0.0;
for (int i = 0; i < v.size(); i++) {
   Object temp = v.elementAt(i);
   if (temp instanceof Double)
       sum + = ((Double) temp).doubleValue();
}
System.out.println("Sum of elements in vector v is " + sum);
```

Note: Improved Version (Modern, Type-Safe and enhanced `for` loop)

```
Vector<Double> v = new Vector<>();
// Add elements to v...

double sum = 0.0;
for (Double num : v) {
    if (num != null) {
        sum += num; // auto-unboxing
    }
}
System.out.println("Sum of elements in vector v is " + sum);
```

Example: Use Vector to Store a Collection of `employee`

VECTOR

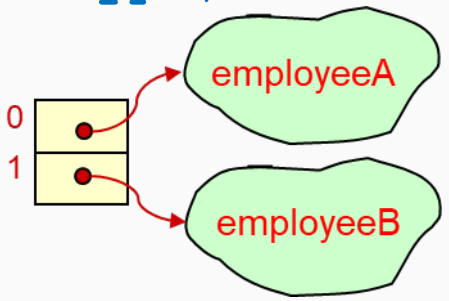
- Import statement: `import java.util.*;`
- Data declaration: `Vector employees = new Vector(); //raw type`

  `Vector<String> employees = new Vector<>(); //generics type, better`
- Add existing objects:

  `employees.addElement(employeeA);`

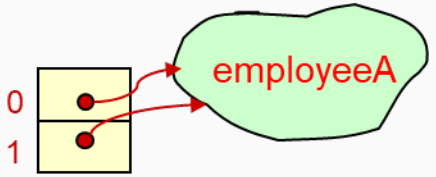
  `employees.addElement(employeeB);`

  OR **`employees.add(employeeA); //if thread safety is not required`**



- Access 2nd element:

  `anEmployee = (Employee)employees.elementAt(1);`



- Change an element:

  `employees.setElementAt(employeeA, 1);`

Example: A Collection of `employee` (Cont.)

VECTOR

- **Insert** an element:

```
employees.insertElementAt(employeeB, 1);
```

- **Change** an element:

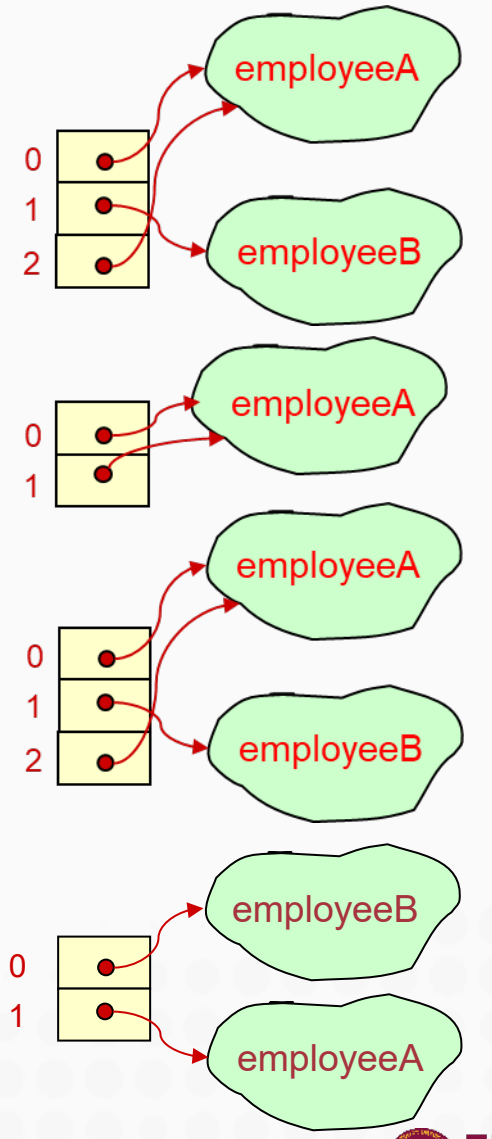
```
employees.setElementAt(employeeA, 1);
```

- **Insert** an element:

```
employees.insertElementAt(employeeB, 1);
```

- **Remove** an element:

```
employees.removeElementAt(0);
```



Methods for `Vector` Class

VECTOR

Method Call	Effect

`Vector employees = new Vector();`
`Vector<Employee> employees = new Vector<>();`
`//generics type, better`
	Construct an empty Vector (must specify the type of elements using generics, or use raw type if generics are not specified – not recommended)

`employees.addElement(emp1);`	Appends object `emp1` to vector `employees` and increments its size

`(Employee)employees.elementAt(0);`
`Employee emp = employees.elementAt(0);`
`//no cast required for generics data type`
	Returns the object at index 0 and converts it to type `Employee`

`employees.setElementAt(emp2, 1)`	Changes the object at index 1 to `emp2`
`employees.insertElementAt(anEmp, 1);`	Insert object `anEmp` at index 1, shifting the elements that follow up one position. Increment vector size.
`employees.removeElementAt(1);`	Remove the element at index 1, shifting the elements that follow down one position. Decrement vector size.
`employees.toString();`	Forms a string enclosed in square brackets. The objects in `employees` are separated by commas.

Vector: Example 3

VECTOR

```java
import java.util.Vector;

// Employee class definition
class Employee {
    private String name;
    private int id;

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public String toString() {
        return "Employee{id=" + id + ", name='" + name +
"'}";
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        // Create a Vector of Employees
        Vector<Employee> employees = new Vector<>();

        // Add employees
        employees.add(new Employee("Alice", 101));
        employees.add(new Employee("Bob", 102));
        employees.add(new Employee("Charlie", 103));

        System.out.println("After adding employees:");
        printVector(employees);
```

Vector: Example 3 (Cont.)

VECTOR

```
        // Insert an employee at index 1
        employees.insertElementAt(new Employee("Diana", 104), 1);

        System.out.println("\nAfter inserting Diana at index 1:");
        printVector(employees);

        // Access an employee
        Employee e = employees.get(2);
        System.out.println("\nEmployee at index 2: " + e);

        // Replace an employee at index 0
        employees.set(0, new Employee("Eve", 105));
        System.out.println("\nAfter replacing employee at index 0 with
Eve:");
        printVector(employees);

        // Remove by index
        employees.removeElementAt(1);
        System.out.println("\nAfter removing employee at index 1:");
        printVector(employees);
```

```
        // Remove by object
        Employee toRemove = employees.get(1);
        employees.remove(toRemove);
        System.out.println("\nAfter removing employee " + toRemove +
" by object:");
        printVector(employees);

        // Size of the vector
        System.out.println("\nNumber of employees: " +
employees.size());
    }

    // Helper method to print all employees
    public static void printVector(Vector<Employee> employees) {
        for (Employee emp : employees) {
            System.out.println(emp);
        }
    }
}
```

VECTOR

Array vs. ArrayList vs. Vector

- Static - fixed size:

```
int anArray[ ] = {10, 20, 30};
```

OR `Employee emp[] = new Employee[3];`

- Dynamic - unlimited size:

```
ArrayList empList = new ArrayList();
```

OR `ArrayList<String> empList = new ArrayList<>();`

- Dynamic - unlimited size, **synchronized** for **thread safety** :

```
Vector empList = new Vector();
```

OR `Vector<String> empList = new Vector<>();`